

ISPGAYA

instituto superior politécnico

Escola Superior de Ciência e Tecnologia
Licenciatura em Engenharia Informática - 2024/2025
Linguagens e Teorias da Computação



RENT CAR LDA
L T C

Aluno:
Miguel Magalhães (2021103166)

Docente: Sr. Prof. José Arnaud

20 de Dezembro de 2024

ÍNDICE DE CONTEÚDOS

Índice de Conteúdos.....	2
Índice de Imagens	4
Abreviaturas.....	6
Glossário	7
Introdução	8
Enquadramento	10
Objetivos	12
Desenvolvimento de uma aplicação funcional para gestão de aluguer de automóveis.....	12
Implementação de funcionalidades de gestão de automóveis e clientes	12
Gestão de alugueres e cálculo do valor total	13
Validação e integridade dos dados.....	13
Geração de relatórios e listagens.....	13
Criação de uma interface intuitiva e amigável para o utilizador.....	13
Implementação de boas práticas de programação e design de software.....	14
Testar e garantir a robustez da aplicação	14
Estrutura do Relatório	15
Desenvolvimento	17
Desenvolvimento.....	17
1. Planeamento e Arquitetura do Sistema	17
2. Implementação das Funcionalidades Principais	17
2.1. Gestão de Automóveis	18
2.2. Gestão de Clientes.....	18
2.3. Gestão de Alugueres	18
2.4. Listagens e Relatórios.....	19
2.5. Interface com o Utilizador	19

3. Testes e Validação.....	19
Considerações Finais sobre o Desenvolvimento.....	20
Manual do Utilizador:	21
Requisitos do Sistema.....	21
Hardware Recomendado:	21
Instalação e Configuração	21
Verificação do Java:	22
Iniciando a Aplicação	22
Descrição das Funcionalidades Principais	23
Análise do Código/ Estrutura de Dados	29
Classe Automovel.java	29
Classe Cliente.java.....	31
Classe Aluguer.java.....	33
AluguerService.java	36
AutomovelService.java.....	38
ClienteService.java	39
Pasta utils	41
Análise do Arquivo Main.java.....	47
Conclusão.....	53
Bibliografia	55
Anexos	57

ÍNDICE DE IMAGENS

Figura i: Verificar a versão do java instalado.....	22
Figura ii: Tela inicial.....	22
Figura iii: Atributos.	29
Figura iv: Getters e Setters.....	29
Figura v: Construtor.....	30
Figura vi: Método toString()......	30
Figura vii: Atributos da Classe Cliente.....	31
Figura viii: Métodos da Classe Cliente.....	31
Figura ix: Construtor da Classe Cliente.....	32
Figura x: Método da Classe Cliente.....	32
Figura xi: Atributos da Classe Aluguer.	33
Figura xii: Métodos da Classe Aluguer.	33
Figura xiii: Construtor da Classe Aluguer.	34
Figura xiv: Método calcularValor()......	34
Figura xv: Método toString()......	34
Figura xvi: Método adicionarAluguer(Aluguer aluguer).....	36
Figura xvii: Exemplo intuitivo à explicação.....	37
Figura xviii: Método listarAutomoveisDisponiveis(List<Automovel> todosAutomoveis).....	37
Figura xix: Método adicionarAutomovel(Automovel automovel).....	38
Figura xx: Exemplo intuitivo da explicação acima.	39
Figura xxi: Método removerAutomovel(String matricula).	39
Figura xxii: Método adicionarCliente(Cliente cliente).....	39
Figura xxiii: Método removerCliente(String cc).	40
Figura xxiv. Atributos da Classe InputUtils.	41
Figura xxv: Implementação dos métodos referidos acima.	42
Figura xxvi: Exemplo do uso do lerString.....	42
Figura xxvii: Implementação de lerInt.....	43
Figura xxviii: Exemplo do uso de lerInt.....	43
Figura xxix: Implementação do lerDouble.	43
Figura xxx: Exemplo de uso do lerDouble.	44

Figura xxxi: Implementação do lerData.	44
Figura xxxii: Funcionamento do lerData.	44
Figura xxxiii: Implementação do fecharScanner().	45
Figura xxxiv: Exemplo de uso decharScanner().	45
Figura xxxv: Flexibilidade no formato da data.	45
Figura xxxvi: Incluir métodos para simular entradas em testes unitários.	46
Figura xxxvii: Importações dos modelos.	47
Figura xxxviii:	47
Figura xxxix: Menu Principal do output.	48
Figura xl: Opções do Menu.	48
Figura xli: Métodos Auxiliares: <code>gerirAutomoveis(AutomovelService automovelService)</code>	49
Figura xlii: <code>gerirClientes(ClienteService clienteService)</code>	49
Figura xliii: Automóveis alugados.	50
Figura xliv: Automóveis disponíveis.	50
Figura xlv: Alugueres ordenados por data de entrega.	50

ABREVIATURAS

- **CC:** Cartão de Cidadão, identificação oficial única do cliente.
- **MVC:** Model-View-Controller, padrão de arquitetura utilizado implicitamente para modularização.
- **SQL:** Structured Query Language, mencionada como possível futura implementação de banco de dados.
- **NoSQL:** Alternativa a SQL para armazenar dados não relacionais, também considerada como futura extensão.
- **API:** Application Programming Interface, mencionada para integração futura.

GLOSSÁRIO

- **Matrícula:** Identificador único de cada automóvel, equivalente à placa de registo.
- **Menu principal:** Interface inicial do sistema, onde o utilizador escolhe as ações desejadas.
- **Orientação a objetos:** Paradigma de programação usado no projeto para modelar os elementos do sistema.
- **Serviços (Services):** Componentes responsáveis por encapsular a lógica de negócios, como gestão de clientes, automóveis e alugueres.
- **Validação:** Processo de verificar a integridade e a conformidade das entradas no sistema.

INTRODUÇÃO

O presente relatório descreve o desenvolvimento e a implementação de um sistema para a gestão do aluguer de automóveis, no âmbito da unidade curricular de Linguagens e Teoria da Computação do curso de Engenharia Informática, na Escola Superior de Ciência e Tecnologia, no ano letivo de 2024/2025. Este trabalho foi realizado no contexto de uma proposta apresentada pelo professor José Arnaud, que visava a criação de uma aplicação para gerir os processos de aluguer de veículos, atendendo tanto à gestão de automóveis e clientes quanto ao registo e controlo dos alugueres. A aplicação foi desenvolvida utilizando a linguagem de programação Java, um dos pilares para a realização de sistemas robustos e eficientes, e visa automatizar e otimizar processos, garantindo maior segurança e eficiência operacional para a empresa fictícia “Rent Car Lda”.

O sistema desenvolvido tem como objetivo principal permitir o controlo de automóveis disponíveis para aluguer, a gestão de clientes, a realização de alugueres com o cálculo automático do valor a pagar e a possibilidade de gerir e visualizar informações sobre a frota de veículos e os alugueres realizados. Além disso, a aplicação foi projetada para ser capaz de realizar operações de inserção, alteração e remoção de informações, assegurando a integridade da base de dados. A aplicação também valida todas as interações dos utilizadores com o sistema, como a verificação de datas e a impossibilidade de realizar alugueres de veículos já alugados.

Este projeto foi desenvolvido em Java, devido à sua robustez, versatilidade e grande aplicabilidade no desenvolvimento de sistemas empresariais e aplicações de gestão. Java permite a criação de sistemas modulares e bem estruturados, que são de fácil manutenção e extensibilidade. O sistema implementado faz uso de conceitos fundamentais de orientação a objetos, como classes, herança e encapsulamento, para garantir um código mais organizado e reutilizável. Além disso, a arquitetura do projeto seguiu boas práticas de desenvolvimento de software, como a separação de responsabilidades e a definição clara de serviços e camadas.

O sistema oferece uma interface de linha de comando (CLI), com menus interativos que permitem aos utilizadores interagir com o sistema de forma intuitiva. Além disso, a implementação inclui a validação de entradas, para garantir que as operações realizadas sejam consistentes e seguras. Para a gestão de dados, foram utilizadas estruturas de dados

adequadas para cada tipo de informação, e o sistema foi projetado para garantir a integridade das informações, evitando erros como a duplicação de dados e a criação de alugueres com automóveis não existentes ou em conflito.

O trabalho realizado nesta unidade curricular teve como principais objetivos aprofundar os conhecimentos em linguagens de programação e teoria da computação, com ênfase no uso da linguagem Java, na gestão de dados e na construção de sistemas robustos. Este relatório detalha todos os aspetos técnicos e metodológicos envolvidos no desenvolvimento da aplicação, bem como as dificuldades enfrentadas e as soluções encontradas. Além disso, são discutidos os requisitos do sistema e os algoritmos utilizados para garantir o bom funcionamento do programa.

ENQUADRAMENTO

O desenvolvimento de sistemas de software destinados à gestão de processos empresariais, como o aluguer de automóveis, exige a aplicação de conceitos sólidos em programação, estruturação de dados e modelagem de processos de negócios. Este projeto, que visa criar uma aplicação para a gestão do aluguer de uma frota de veículos na empresa fictícia “Rent Car Lda”, encontra-se enquadrado da unidade curricular de Linguagens e Teoria da Computação, parte integrante do curso de Engenharia Informática da Escola Superior de Ciência e Tecnologia. O objetivo da unidade curricular é fornecer aos alunos uma base sólida em teoria da computação, além de desenvolver competências práticas na implementação de sistemas de software, utilizando linguagens de programação como Java.

A escolha do tema “Gestão do Aluguer de uma Frota de Automóveis” é particularmente relevante, uma vez que a gestão de automóveis e o processo de aluguer envolvem múltiplas variáveis e a necessidade de uma estrutura bem definida para controlar as operações diárias. O mercado de aluguer de automóveis é altamente competitivo e, para que uma empresa mantenha a sua eficiência e sustentabilidade, é essencial que os seus processos sejam bem geridos e otimizados. A informatização de processos empresariais como o controlo de alugueres, gestão de frotas e clientes, e o cálculo de valores a pagar, permite aumentar a precisão e a produtividade dos funcionários, ao mesmo tempo, em que proporciona uma experiência mais fluida e eficiente para os utilizadores.

Neste contexto, a aplicação desenvolvida neste trabalho simula uma solução que poderia ser adotada por uma empresa de aluguer de automóveis, visando resolver problemas comuns na gestão dessa área, como o controlo de veículos disponíveis para aluguer, a verificação de datas conflitantes nos alugueres e a impossibilidade de registar alugueres de automóveis já alugados. Além disso, a validação dos dados e a integridade da informação são aspetos fundamentais para garantir o bom funcionamento de um sistema de gestão. O desenvolvimento desta aplicação não só aborda os desafios da gestão de uma frota de veículos, mas também permite aplicar e consolidar conceitos importantes da teoria da computação, como algoritmos, estruturas de dados e organização de software.

O enquadramento teórico para o desenvolvimento deste sistema abrange várias áreas da ciência da computação, incluindo a programação orientada a objetos (POO), a manipulação de dados estruturados e a utilização de algoritmos eficientes para resolver problemas práticos. A utilização de Java como linguagem de programação é fundamental,

uma vez que a sua robustez, a facilidade de manutenção e a flexibilidade são ideais para a criação de sistemas complexos e modulares. A POO, por exemplo, é aplicada para estruturar as diferentes entidades do sistema, como automóveis, clientes e alugueres, em classes e objetos, permitindo uma gestão eficiente e flexível dos dados.

Este trabalho também está diretamente ligado ao contexto de gestão empresarial, onde a informática desempenha um papel crucial na otimização dos processos operacionais. A automatização de tarefas, como o cálculo do valor dos alugueres, a verificação de consistência das datas e a gestão da informação sobre os automóveis e clientes, visa não só aumentar a eficiência, mas também reduzir o risco de erros humanos, melhorando assim a fiabilidade do sistema. Além disso, o trabalho envolve a criação de uma interface de utilizador que seja intuitiva e de fácil utilização, permitindo que tanto os clientes como os funcionários possam interagir com o sistema de forma eficiente.

O desenvolvimento desta aplicação também inclui a implementação de funcionalidades específicas de pesquisa e listagem de dados, essenciais para a organização e o controlo de grandes volumes de informação. A possibilidade de pesquisar por automóveis, clientes e datas, bem como de gerar listagens de veículos alugados e não alugados, oferece ao utilizador uma visão clara e detalhada da operação da empresa. A implementação desses recursos é fundamental para garantir que a aplicação seja não apenas funcional, mas também fácil de usar e capaz de lidar com diferentes cenários que possam surgir no quotidiano da empresa de aluguer de automóveis.

OBJETIVOS

O principal objetivo deste projeto é o desenvolvimento de uma aplicação de software para a gestão de aluguer de uma frota de automóveis na empresa fictícia “Rent Car Lda”. Esta aplicação visa resolver uma série de desafios que surgem no contexto da gestão de frotas e no controlo dos alugueres de veículos. A seguir, são apresentados os objetivos principais do projeto:

DESENVOLVIMENTO DE UMA APLICAÇÃO FUNCIONAL PARA GESTÃO DE ALUGUER DE AUTOMÓVEIS

O primeiro e mais importante objetivo deste trabalho é criar uma aplicação que seja capaz de gerir de forma eficiente todos os aspetos envolvidos no processo de aluguer de automóveis. A aplicação deve permitir o registo, consulta, alteração e eliminação de dados relacionados aos automóveis, clientes e alugueres. Isso inclui o controlo do estado dos veículos (se estão ou não disponíveis para aluguer), a verificação de datas de alugueres e a manutenção da integridade dos dados ao longo do processo.

IMPLEMENTAÇÃO DE FUNCIONALIDADES DE GESTÃO DE AUTOMÓVEIS E CLIENTES

A aplicação deverá permitir o registo e a gestão de informações sobre os automóveis e clientes da empresa. Para os automóveis, as funcionalidades incluem o registo de dados como marca, modelo, cor, ano de aquisição, matrícula e valor por dia. Para os clientes, a aplicação deve permitir o registo de dados pessoais como nome, morada, número de cartão de cidadão e carta de condução. Estas informações devem ser acessíveis e editáveis, permitindo o controlo completo dos dados relevantes para o processo de aluguer.

GESTÃO DE ALUGUERES E CÁLCULO DO VALOR TOTAL

Outro objetivo crucial deste projeto é permitir o registo e a gestão de alugueres. A aplicação deve garantir que não seja possível alugar um automóvel que já esteja alugado e que as datas de início e fim do aluguer sejam válidas. Além disso, deve ser capaz de calcular automaticamente o valor total a pagar pelo cliente, baseado na duração do aluguer e no valor diário do automóvel.

VALIDAÇÃO E INTEGRIDADE DOS DADOS

A aplicação deve ser capaz de realizar a validação dos dados inseridos. Por exemplo, deve garantir que as datas de fim de aluguer não sejam anteriores às datas de início e que os alugueres não sejam registados para automóveis que não existem ou estão indisponíveis. Essas validações são fundamentais para assegurar que o sistema funcione de forma eficaz, evitando erros operacionais que poderiam comprometer a fiabilidade da gestão.

GERAÇÃO DE RELATÓRIOS E LISTAGENS

A aplicação também deverá fornecer funcionalidades para gerar relatórios e listagens úteis para a administração da empresa. Estas listagens devem incluir informações sobre os automóveis alugados e não alugados, clientes e alugueres. Além disso, a aplicação deve permitir pesquisas específicas por automóveis, clientes ou datas, o que torna o sistema mais flexível e adaptável às necessidades do utilizador.

CRIAÇÃO DE UMA INTERFACE INTUITIVA E AMIGÁVEL PARA O UTILIZADOR

Um objetivo fundamental do projeto é o desenvolvimento de uma interface de utilizador (UI) que seja clara, intuitiva e fácil de usar. A interface deve permitir que o utilizador interaja com o sistema de forma eficiente, sem a necessidade de formação prévia. A experiência do utilizador (UX) deve ser cuidadosamente desenhada para garantir que as tarefas mais comuns, como registar alugueres ou consultar a disponibilidade dos automóveis, possam ser feitas de forma rápida e sem erros.

IMPLEMENTAÇÃO DE BOAS PRÁTICAS DE PROGRAMAÇÃO E DESIGN DE SOFTWARE

Este projeto também visa aplicar boas práticas de programação, como a utilização da programação orientada a objetos (POO), modularização do código e organização adequada dos ficheiros e pastas. A aplicação deve ser construída de forma escalável e manutenível, de modo que futuras modificações ou expansões possam ser feitas com facilidade, sem comprometer a qualidade do código.

TESTAR E GARANTIR A ROBUSTEZ DA APLICAÇÃO

Outro objetivo importante deste projeto é garantir que a aplicação seja robusta e livre de falhas críticas. Para isso, serão realizados testes em diferentes cenários para garantir que a aplicação funcione conforme as expectativas e que erros sejam tratados adequadamente. A robustez do sistema será fundamental para assegurar que a empresa “Rent Car Lda” possa confiar no sistema para a gestão diária dos seus processos.

ESTRUTURA DO RELATÓRIO

O presente relatório está estruturado em sete capítulos principais, visando detalhar a análise e o desenvolvimento do sistema de gestão de aluguer de automóveis. Este relatório abrange a arquitetura do software, a análise detalhada de cada classe do sistema, as melhorias propostas, e as conclusões gerais sobre o processo de desenvolvimento.

No **Capítulo 1**, são apresentados os objetivos do projeto, que têm como foco o desenvolvimento de um sistema capaz de gerir o aluguer de automóveis, clientes e alugueres, utilizando conceitos de programação orientada a objetos e boas práticas de design de software. Este capítulo descreve o escopo do projeto, destacando as principais funcionalidades implementadas, como a gestão de automóveis, clientes e alugueres, bem como a interação do utilizador mediante menus e validações.

O **Capítulo 2** detalha as classes principais que compõem o sistema, como **Automovel.java**, **Cliente.java**, e **Aluguer.java**, e os serviços associados (**AluguerService.java**, **AutomovelService.java**, e **ClienteService.java**). Para cada classe, são descritos os seus atributos e métodos, com destaque para a funcionalidade e o papel de cada uma na arquitetura do sistema. São também apresentadas sugestões de melhorias, como a validação de matrículas, a inclusão de cálculos de valores de aluguer, e a implementação de métodos de comparação, além da proposta de otimização dos processos de busca e verificação de disponibilidade de veículos.

No **Capítulo 3**, a análise do arquivo **InputUtils.java** é realizada, destacando a importância dessa classe como utilitário para validação e leitura de entradas do utilizador, como strings, inteiros, números decimais e datas. O capítulo sugere melhorias na flexibilidade do formato de data e na adaptação da classe para outras fontes de entrada, além de discutir a estabilidade e a segurança da aplicação, enfatizando a importância de uma abordagem padronizada para lidar com a interação do utilizador.

O **Capítulo 4** foca na análise do arquivo **Main.java**, que constitui o ponto de entrada do sistema. Aqui, é discutida a estrutura modular do código, que organiza as operações principais em métodos dedicados para a gestão de automóveis, clientes e alugueres. Também são abordadas as validações de entrada e a clareza do menu interativo, com sugestões para melhorar o encapsulamento das opções de menu e implementar novos relatórios, como estatísticas de alugueres por cliente e receita total.

No **Capítulo 5**, são feitas as sugestões de melhorias gerais, como a aplicação de padrões de design, como o **Singleton** e o **Factory Method**, a fim de melhorar a gestão das instâncias dos serviços e a criação dos objetos. A persistência de dados é outro ponto destacado, sugerindo que a implementação de uma base de dados, seja SQL ou NoSQL, garantiria a persistência das informações. A validação de entradas e a implementação de erros padronizados são igualmente apontadas como melhorias a serem adotadas.

No **Capítulo 6**, são abordados os desafios enfrentados durante o desenvolvimento do sistema, como a complexidade das validações de regras de negócios, o cuidado na integração entre componentes e a estruturação do menu para garantir uma navegação acessível e intuitiva. A análise crítica também enfatiza a importância da modularidade, da validação e do feedback do utilizador, garantindo uma aplicação robusta e fácil de manter.

Por fim, no **Capítulo 7**, são apresentadas as conclusões do projeto, destacando os principais aprendizados adquiridos ao longo do desenvolvimento, como a aplicação de conceitos de programação orientada a objetos, boas práticas de design e validação de entradas. O relatório conclui com sugestões de melhorias e extensões, como a integração com uma base de dados, a evolução para uma interface gráfica utilizando JavaFX ou Swing, e a implementação de relatórios financeiros detalhados. O sistema desenvolvido serve como uma base sólida para evoluções futuras e representa um marco significativo na trajetória académica e profissional.

Este relatório segue uma estrutura coesa, que abrange tanto os aspetos técnicos quanto as possíveis melhorias, e foi elaborado visando fornecer uma análise detalhada do processo de desenvolvimento do sistema de gestão de aluguer de automóveis, aplicando conceitos fundamentais da Engenharia Informática.

DESENVOLVIMENTO

DESENVOLVIMENTO

O desenvolvimento deste projeto envolveu diversas fases, incluindo o planeamento, a conceção do design da aplicação, a implementação do código e, finalmente, a realização de testes para garantir a robustez e a funcionalidade do sistema. A aplicação foi desenvolvida utilizando a linguagem de programação Java, o que permitiu a utilização de diversos recursos para garantir a eficiência e a flexibilidade do sistema. A seguir, são descritos os principais aspetos que envolveram o desenvolvimento da aplicação.

1. PLANEAMENTO E ARQUITETURA DO SISTEMA

Antes de iniciar a implementação, foi necessário definir a arquitetura do sistema e planear como as funcionalidades seriam distribuídas e implementadas. A arquitetura adotada para o projeto seguiu os princípios da **Programação Orientada a Objetos (POO)**, o que permitiu a criação de classes que modelam as entidades principais do sistema, como **Automóvel**, **Cliente** e **Aluguer**.

A escolha de utilizar a POO foi baseada na necessidade de manter o código organizado, modular e fácil de manter. Cada entidade do sistema foi mapeada para uma classe com atributos e métodos específicos, permitindo que o código fosse escalável e reutilizável. Além disso, foi tomada a decisão de utilizar o **padrão de design MVC** (Modelo-Visão-Controlador) para separar as responsabilidades do sistema e garantir que a interface com o utilizador estivesse desacoplada da lógica de negócio e da manipulação de dados.

2. IMPLEMENTAÇÃO DAS FUNCIONALIDADES PRINCIPAIS

A implementação do projeto envolveu o desenvolvimento das funcionalidades principais da aplicação, cruciais para o funcionamento do sistema. As funcionalidades foram divididas em várias secções, sendo que as mais importantes são as seguintes:

2.1. *GESTÃO DE AUTOMÓVEIS*

A gestão de automóveis foi uma das primeiras funcionalidades implementadas. Para isso, foi criada uma classe **Automovel**, que contém atributos como **marca, modelo, cor, ano de aquisição, matrícula e valor por dia**. Através desta classe, o sistema pode realizar operações de inserção, alteração e eliminação de veículos, garantindo a integridade dos dados. Além disso, foi implementada a validação para garantir que um automóvel não seja duplicado no sistema e que as informações inseridas sejam consistentes.

2.2. *GESTÃO DE CLIENTES*

Similar à gestão de automóveis, foi criada a classe **Cliente**, que armazena informações como **nome, morada, número de cartão de cidadão e carta de condução**. A aplicação permite o registo, a atualização e a exclusão de clientes, bem como a validação para garantir que as informações inseridas sejam únicas e corretas.

2.3. *GESTÃO DE ALUGUERES*

A funcionalidade de gestão de alugueres é central para a aplicação, pois envolve a interação entre as classes **Automovel** e **Cliente**. Para isso, foi desenvolvida a classe **Aluguer**, que contém informações sobre o veículo alugado, o cliente, a data de início e a data de fim do aluguer. Uma das principais funcionalidades associadas a esta classe é o cálculo do valor a ser pago pelo cliente, com base no número de dias de aluguer e no valor diário do automóvel. Além disso, a aplicação verifica se o automóvel está disponível nas datas solicitadas e se a data de fim do aluguer não é inferior à data de início.

Foi dado especial cuidado à validação de dados nesta parte do sistema, para evitar que um automóvel fosse alugado quando estivesse já em aluguer ou que um aluguer fosse registado com datas inválidas. O sistema também garante que o valor do aluguer seja calculado corretamente, levando em consideração o número de dias entre as datas de início e fim.

2.4. *LISTAGENS E RELATÓRIOS*

A aplicação inclui funcionalidades que permitem ao utilizador gerar relatórios e listagens detalhadas sobre os automóveis, clientes e alugueres. Estas listagens incluem, entre outras, as seguintes:

- **Listagem de automóveis alugados e não alugados:** Permite ao utilizador verificar a disponibilidade de veículos.
- **Listagem de automóveis alugados ordenada por data de entrega:** Facilita o controle dos veículos alugados, organizando-os pela data de devolução.
- **Listagem de clientes e alugueres:** Mostra os clientes registados e os respetivos alugueres.

Além disso, a aplicação permite a pesquisa de automóveis e clientes, bem como a pesquisa por datas de alugueres, o que torna o sistema mais dinâmico e eficaz no momento de consultar informações.

2.5. *INTERFACE COM O UTILIZADOR*

A interface do utilizador foi desenvolvida utilizando uma interface de linha de comando (CLI), proporcionando uma interação direta e eficiente com o sistema. O design da interface foi focado na simplicidade e na funcionalidade, permitindo que os utilizadores interajam com a aplicação de forma fácil e eficiente.

A GUI foi organizada de modo a apresentar as funcionalidades de forma clara, com menus para gestão de automóveis, clientes e alugueres, além de campos de pesquisa e listagens dinâmicas para facilitar o acesso às informações. A navegação entre as diferentes áreas do sistema é fluida, e todas as operações podem ser realizadas com poucos cliques.

3. *TESTES E VALIDAÇÃO*

Após a implementação das funcionalidades principais, foi realizado um processo de testes rigorosos para garantir a fiabilidade e a robustez do sistema. Foram feitos testes unitários para verificar o comportamento das classes e métodos individuais, bem como testes integrados para garantir que as diferentes partes do sistema funcionam em conjunto sem erros.

Além disso, a aplicação foi testada com diferentes cenários de utilizadores e de dados, a fim de identificar potenciais falhas e assegurar que o sistema seja capaz de lidar com situações imprevistas, como a tentativa de registo de dados inválidos ou inconsistentes.

CONSIDERAÇÕES FINAIS SOBRE O DESENVOLVIMENTO

O desenvolvimento deste projeto exigiu uma abordagem estruturada e a utilização de boas práticas de programação. A escolha de Java como linguagem de desenvolvimento e o uso de padrões como a Programação Orientada a Objetos e o design MVC foram fundamentais para garantir a qualidade e a escalabilidade do sistema. Além disso, o desenvolvimento da interface gráfica com JavaFX contribuiu para a criação de uma aplicação moderna e fácil de usar, atendendo às necessidades da empresa fictícia “Rent Car Lda”.

MANUAL DO UTILIZADOR:

Este manual destina-se a orientar os utilizadores na utilização da aplicação de gestão de aluguer de automóveis desenvolvida para a empresa fictícia “Rent Car Lda”. Nele, serão apresentadas as funcionalidades principais, os passos para executar as operações básicas e soluções para problemas comuns que possam surgir durante o uso da aplicação.

REQUISITOS DO SISTEMA

- **Software Necessário:**
 - **Java Runtime Environment (JRE) 8 ou superior:**
Necessário para executar a aplicação.
 - **Sistema Operacional:** Windows, macOS ou Linux.

HARDWARE RECOMENDADO:

- **Processador:** Dual-core 2.0 GHz ou superior.
- **Memória RAM:** Mínimo de 4 GB.
- **Espaço em Disco:** Mínimo de 100 MB para instalação e operação.

As informações acima foram retiradas através de uma documentação oficial da Oracle: <https://docs.oracle.com/cd/E19226-01/821-1337/abpaj/index.html> .

INSTALAÇÃO E CONFIGURAÇÃO

Instruções para Acesso ao Código-Fonte:

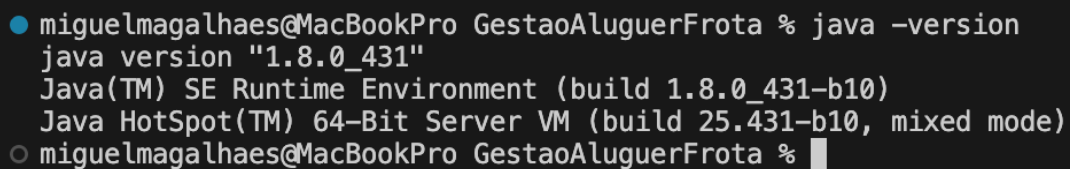
Para acessar o código-fonte da aplicação, siga os passos abaixo:

1. **Descompactar o Arquivo:** Extraia o conteúdo do arquivo .rar utilizando uma ferramenta de descompactação, como o WinRAR ou 7-Zip.
2. **Abrir a Pasta de Código:** Navegue até a pasta descompactada e abra a pasta intitulada “código”.

3. **Abrir no Visual Studio Code (VSC):** Dentro da pasta “código”, abra o projeto no Visual Studio Code, que é o framework que foi utilizado para este projeto. Embora seja possível testar o código em outro framework, isso pode exigir ajustes adicionais e potencialmente comprometer a funcionalidade do trabalho desenvolvido.

VERIFICAÇÃO DO JAVA:

- Certifique-se de que o Java está instalado no seu sistema. Para verificar, abra o terminal ou prompt de comando e execute:



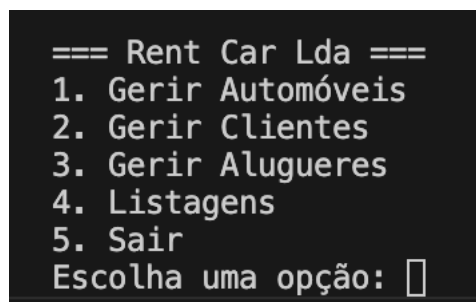
```
● miguelmagalhaes@MacBookPro GestaoAluguerFrota % java -version
java version "1.8.0_431"
Java(TM) SE Runtime Environment (build 1.8.0_431-b10)
Java HotSpot(TM) 64-Bit Server VM (build 25.431-b10, mixed mode)
○ miguelmagalhaes@MacBookPro GestaoAluguerFrota %
```

Figura i: Verificar a versão do java instalado.

- Se o Java não estiver instalado, faça o download e a instalação a partir do [site oficial da Oracle](#).

INICIANDO A APLICAÇÃO

- **Tela Inicial:**
 - Ao iniciar a aplicação, será exibido o **Menu Principal**, onde o utilizador pode escolher entre diferentes opções de gestão:



```
=== Rent Car Lda ===
1. Gerir Automóveis
2. Gerir Clientes
3. Gerir Alugueres
4. Listagens
5. Sair
Escolha uma opção: █
```

Figura ii: Tela inicial.

DESCRIÇÃO DAS FUNCIONALIDADES PRINCIPAIS

Gestão de Automóveis

- **Adicionar Automóvel:**

1. Selecione a opção 1 no Menu Principal e, em seguida, 1 novamente para adicionar um automóvel.
2. Preencha os campos solicitados:
 - **Marca:** Exemplo: Toyota
 - **Modelo:** Exemplo: Corolla
 - **Cor:** Exemplo: Azul
 - **Ano de Aquisição:** Exemplo: 2020
 - **Matrícula:** Exemplo: ABC-1234
 - **Valor por Dia:** Exemplo: 50.00
3. Confirme a adição. Se a matrícula já existir, uma mensagem de erro será exibida.

- **Alterar Automóvel:**

1. Selecione a opção 1 no Menu Principal e, em seguida, 2 para alterar um automóvel.
2. Insira a matrícula do automóvel que deseja alterar.
3. Atualize os campos desejados.
4. Confirme as alterações.

- **Remover Automóvel:**

1. Selecione a opção 1 no Menu Principal e, em seguida, 3 para remover um automóvel.
2. Insira a matrícula do automóvel que deseja remover.
3. Confirme a remoção. Se o automóvel estiver associado a um aluguer ativo, a remoção será impedida.

- **Listar Automóveis:**

- Exibe a lista completa de automóveis cadastrados, indicando quais estão disponíveis ou alugados.

Gestão de Clientes

- **Adicionar Cliente:**

1. Selecione a opção 2 no Menu Principal e, em seguida, 1 para adicionar um cliente.
2. Preencha os campos solicitados:
 - **Nome:** Exemplo: João Silva
 - **Morada:** Exemplo: Rua das Flores, 123
 - **CC:** Exemplo: 123456789
 - **Carta de Condução:** Exemplo: B1234567
3. Confirme a adição. Se o CC já existir, uma mensagem de erro será exibida.

- **Alterar Cliente:**

1. Selecione a opção 2 no Menu Principal e, em seguida, 2 para alterar um cliente.
2. Insira o número do CC do cliente que deseja alterar.
3. Atualize os campos desejados.
4. Confirme as alterações.

- **Remover Cliente:**

1. Selecione a opção 2 no Menu Principal e, em seguida, 3 para remover um cliente.
2. Insira o número do CC do cliente que deseja remover.
3. Confirme a remoção. Se o cliente tiver alugueres pendentes, a remoção será impedida.

- **Listar Clientes:**

- Exibe a lista completa de clientes cadastrados.

Gestão de Alugueres

- **Adicionar Aluguer:**

1. Selecione a opção 3 no Menu Principal e, em seguida, 1 para adicionar um aluguer.
2. Insira a matrícula do automóvel que deseja alugar.
3. Insira o número do CC do cliente.
4. Insira as datas de início e fim do aluguer (formato: dd/MM/yyyy).
5. O sistema calculará automaticamente o valor total a pagar.
6. Confirme a adição do aluguer. Se o automóvel estiver indisponível nas datas selecionadas, uma mensagem de erro será exibida.

- **Alterar Aluguer:**

1. Selecione a opção 3 no Menu Principal e, em seguida, 2 para alterar um aluguer.
2. Insira o ID do aluguer que deseja alterar.
3. Atualize os campos desejados.
4. Confirme as alterações.

- **Remover Aluguer:**

1. Selecione a opção 3 no Menu Principal e, em seguida, 3 para remover um aluguer.
2. Insira o ID do aluguer que deseja remover.
3. Confirme a remoção.

- **Listar Alugueres:**

- Exibe a lista completa de alugueres cadastrados, indicando os automóveis alugados e as suas datas de devolução.

Geração de Relatórios

- **Automóveis Alugados e Não Alugados:**

- Exibe listas separadas de automóveis que estão atualmente alugados e disponíveis para aluguer.

- **Automóveis Alugados Ordenados por Data de Entrega:**

- Exibe uma lista de automóveis alugados, ordenados pela data prevista de devolução.

- **Cientes e Alugueres:**

- Exibe uma relação de clientes e os seus respetivos alugueres.

Pesquisas

- **Pesquisa por Automóveis:**

- Permite buscar automóveis específicos por matrícula, marca ou modelo.

- **Pesquisa por Clientes:**

- Permite buscar clientes por nome ou número de CC.

- **Pesquisa por Datas:**

- Permite buscar alugueres que ocorreram ou ocorrerão em determinadas datas.

Exemplos de Uso

- ***Exemplo 1: Adicionar um Novo Automóvel***

1. No Menu Principal, selecione 1 para Gestão de Automóveis.
2. Selecione 1 novamente para Adicionar Automóvel.

3. Preencha os campos:
 - **Marca:** Toyota
 - **Modelo:** Corolla
 - **Cor:** Azul
 - **Ano de Aquisição:** 2020
 - **Matrícula:** ABC-1234
 - **Valor por Dia:** 50.00
4. Clique em “Salvar”.
5. A aplicação exibirá a mensagem: “Automóvel adicionado com sucesso.”

- **Exemplo 2: Realizar um Aluguer**

1. No Menu Principal, selecione 3 para Gestão de Alugueres.
2. Selecione 1 para Adicionar Aluguer.
3. Insira a matrícula do automóvel: ABC-1234
4. Insira o número do CC do cliente: 123456789
5. Insira a data de início: 15/12/2024
6. Insira a data de fim: 20/12/2024
7. O sistema calculará o valor total: $50.00 * 5 = 250.00$
8. Confirme a adição do aluguer.
9. A aplicação exibirá a mensagem: “Aluguer realizado com sucesso. Valor total: €250.00”

Resolução de Problemas

- **Problema:** A aplicação não inicia.
 - **Solução:** Verifique se o Java está instalado corretamente. Execute “*java -version*” no terminal para confirmar a instalação.
- **Problema:** Erro ao adicionar um automóvel com matrícula duplicada.
 - **Solução:** Verifique se a matrícula já está registada na base de dados. Utilize a opção de listar automóveis para confirmar.

- **Problema:** Não é possível remover um cliente com alugueres ativos.
 - **Solução:** Finalize ou remova os alugueres ativos do cliente antes de tentar removê-lo.

- **Problema:** Erro ao calcular o valor total do aluguer.
 - **Solução:** Verifique se as datas de início e fim estão corretas e no formato adequado (dd/MM/yyyy).

Possíveis Questões

- **Pergunta:** Como atualizar as informações de um cliente?
 - **Resposta:** Acesse a secção de Gestão de Clientes no Menu Principal, selecione 2 para alterar cliente, insira o número do CC do cliente e atualize os campos desejados.

- **Pergunta:** Como gerar uma lista de automóveis disponíveis?
 - **Resposta:** Acesse secção de Geração de Relatórios no Menu Principal e selecione a opção “Automóveis Disponíveis”.

- **Pergunta:** Posso visualizar todos os alugueres realizados por um cliente específico?
 - **Resposta:** Sim, utilize a função de pesquisa por cliente secção de Gestão de Alugueres ou na geração de relatórios para filtrar os alugueres por cliente.

ANÁLISE DO CÓDIGO/ ESTRUTURA DE DADOS

CLASSE AUTOMOVEL.JAVA

Objetivo: Representar os automóveis disponíveis para aluguel no sistema.

Atributos:

- **marca:** Marca do automóvel, como “Toyota”, “BMW”, etc.
- **modelo:** Modelo específico do automóvel, como “Corolla”, “X5”, etc.
- **cor:** Cor do automóvel.
- **cilindrada:** O tamanho do motor, em litros.
- **anoAquisicao:** Ano em que o automóvel foi adquirido ou fabricado.
- **matricula:** A matrícula única do automóvel.
- **valorDia:** O valor do aluguel por dia.

```
private String marca;  
private String modelo;  
private String cor;  
private double cilindrada;  
private int anoAquisicao;  
private String matricula;  
private double valorDia;
```

Figura iii: Atributos.

Métodos: Getters e Setters

São essenciais para acessar e modificar os atributos da classe. Exemplo:

```
public String getMarca() {  
    return marca;  
}  
  
public void setMarca(String marca) {  
    this.marca = marca;  
}
```

Figura iv: Getters e Setters.

Construtor

O construtor permite inicializar os atributos de forma prática, ao criar um novo objeto Automovel.

```
public Automovel(String marca, String modelo, String cor, double cilindrada, int anoAquisicao, String matricula, double valorDia) {  
    this.marca = marca;  
    this.modelo = modelo;  
    this.cor = cor;  
    this.cilindrada = cilindrada;  
    this.anoAquisicao = anoAquisicao;  
    this.matricula = matricula;  
    this.valorDia = valorDia;  
}
```

Figura v: Construtor.

Método toString()

O método toString() é fundamental para fornecer uma representação legível do objeto, o que é útil em exibições no console ou registos.

```
@Override  
public String toString() {  
    return "Marca: " + marca + ", Modelo: " + modelo + ", Cor: " + cor + ", Cilindrada: " + cilindrada +  
        ", Ano: " + anoAquisicao + ", Matrícula: " + matricula + ", Valor por Dia: " + valorDia;  
}
```

Figura vi: Método toString().

Possíveis Melhorias

- **Validação de Matrícula:** Poderíamos incluir validações adicionais, como verificar se a matrícula segue um formato específico.
- **Método de Cálculo de Valor Total de Aluguel:** Adicionar um método que calcule o valor total de um aluguel, considerando o número de dias.

*CLASSE CLIENTE.JAVA***Objetivo**

Representar os clientes que podem alugar automóveis.

Atributos

- **nome:** Nome completo do cliente.
- **morada:** Endereço residencial do cliente.
- **cc:** Número do Cartão de Cidadão (ou qualquer identificação única).
- **cartaConducao:** Número da carta de condução, que pode ser usado para garantir que o cliente está habilitado a alugar um automóvel.

```
private String nome;  
private String morada;  
private String cc; // Cartão de Cidadão (ou número de identificação)  
private String cartaConducao;
```

Figura vii: Atributos da Classe Cliente.

Métodos: Getters e Setters

Permitem o acesso e a modificação dos atributos do cliente.

```
public String getNome() {  
    return nome;  
}  
  
public void setNome(String nome) {  
    this.nome = nome;  
}
```

Figura viii.: Métodos da Classe Cliente.

Construtor

O construtor permite criar um cliente com os dados essenciais.

```
public Cliente(String nome, String morada, String cc, String cartaConducao) {  
    this.nome = nome;  
    this.morada = morada;  
    this.cc = cc;  
    this.cartaConducao = cartaConducao;  
}
```

Figura ix: Construtor da Classe Cliente.

Método toString()

Um método toString() ajuda a visualizar os dados do cliente de forma legível.

```
@Override  
public String toString() {  
    return "Nome: " + nome + ", Morada: " + morada + ", CC: " + cc + ", Carta de Condução: " + cartaConducao;  
}
```

Figura x: Método da Classe Cliente.

Possíveis Melhorias

- **Validação de CC e Carta de Condução:** Incluir validações que garantam que o número do Cartão de Cidadão e da carta de condução estejam no formato correto.
- **Métodos de Comparação:** Poderia ser interessante implementar métodos como equals() para comparar clientes com base no CC, para garantir que não há duplicação no banco de dados.

*CLASSE ALUGUER.JAVA***Objetivo**

Representar um aluguel realizado por um cliente, ligando o Cliente a um Automovel.

Atributos

- **automovel:** O automóvel alugado.
- **cliente:** O cliente que está alugando o automóvel.
- **dataInicio:** A data de início do aluguel.
- **dataFim:** A data de fim do aluguel.

```
private Automovel automovel;  
private Cliente cliente;  
private LocalDate dataInicio;  
private LocalDate dataFim;
```

Figura xi: Atributos da Classe Aluguer.

Métodos: Getters e Setters

Para acessar e modificar os dados do aluguel.

```
public Automovel getAutomovel() {  
    return automovel;  
}  
  
public void setAutomovel(Automovel automovel) {  
    this.automovel = automovel;  
}
```

Figura xii: Métodos da Classe Aluguer.

Construtor

Inicializa os atributos necessários para um aluguel.

```
public Aluguer(Automovel automovel, Cliente cliente, LocalDate dataInicio, LocalDate dataFim) {  
    this.automovel = automovel;  
    this.cliente = cliente;  
    this.dataInicio = dataInicio;  
    this.dataFim = dataFim;  
}
```

Figura xiii: Construtor da Classe Aluguer.

Método calcularValor()

Calcula o valor do aluguel, com base no número de dias.

```
public double calcularValor() {  
    long dias = ChronoUnit.DAYS.between(dataInicio, dataFim);  
    return dias * automovel.getValorDia();  
}
```

Figura xiv: Método calcularValor().

Método toString()

Formata os dados do aluguel para exibição.

```
@Override  
public String toString() {  
    return "Cliente: " + cliente.getNome() + ", Automóvel: " + automovel.getMarca() + " " + automovel.getModelo() +  
    ", Data de Início: " + dataInicio + ", Data de Fim: " + dataFim + ", Valor Total: " + calcularValor();  
}
```

Figura xv: Método toString().

Possíveis Melhorias

- **Validação das Datas:** Garantir que a data de início não seja posterior à data de fim.
- **Métodos de Comparação:** Considerar a implementação de um método equals() baseado na matrícula do automóvel e no CC do cliente.

Conclusão da Pasta models

Essas classes são fundamentais para o funcionamento da aplicação, representando entidades-chave no sistema de aluguel de automóveis. Elas são bem estruturadas, mas há sempre espaço para melhorias, como a adição de validações mais complexas ou métodos auxiliares para manipulação de dados.

ALUGUERSERVICE.JAVA

O serviço de AluguerService tem como função principal a gestão dos alugueres de automóveis. A seguir, vamos detalhar cada um dos métodos e as suas responsabilidades.

Atributo - `private final List<Aluguer> alugueres = new ArrayList<>();`

Esse atributo mantém uma lista de todos os alugueres registados. Ele é inicializado como um ArrayList, que é uma implementação de lista dinâmica em Java, adequada para inserções e buscas rápidas. No entanto, ao longo do desenvolvimento, pode-se considerar o uso de outras coleções dependendo do caso, como HashSet ou LinkedList, dependendo das operações necessárias.

Método adicionarAluguer(Aluguer aluguer)

```
public void adicionarAluguer(Aluguer aluguer) {  
    // Verifica se o automóvel está disponível no período  
    boolean jaAlugado = alugueres.stream()  
        .anyMatch(a -> a.getAutomovel().getMatricula().equals(aluguer.getAutomovel().getMatricula()) &&  
            !(aluguer.getDataFim().isBefore(a.getDataInicio()) || aluguer.getDataInicio().isAfter(a.getDataFim())));
```

Figura xvi: Método adicionarAluguer(Aluguer aluguer).

Explicação:

- **stream():** Utiliza o método stream() para converter a lista de alugueres numa sequência de dados, o que permite usar operações funcionais de maneira eficiente.
- **anyMatch(...):** A função anyMatch() percorre a lista e verifica se **qualquer** automóvel no sistema possui a mesma matrícula que o carro sendo alugado e se as datas de aluguel se sobrepõem.
- **Condição de sobreposição de datas:** A condição `!(aluguer.getDataFim().isBefore(a.getDataInicio()) || aluguer.getDataInicio().isAfter(a.getDataFim()))` assegura que as datas do novo aluguel não se sobreponham com alugueis já existentes.
- **Otimização:** Esse método é eficiente para verificar a disponibilidade de carros em tempo real, mas à medida que a lista de alugueis aumenta, o desempenho

pode ser prejudicado, pois o código percorre todos os alugueis já realizados. Uma possível melhoria seria utilizar uma estrutura de dados como `Map<String, List<Aluguer>>` para indexar os alugueis por matrícula de automóvel, o que permitiria uma busca mais eficiente.

```
if (jaAlugado) {  
    throw new IllegalArgumentException("O automóvel já está alugado no período especificado.");  
}  
  
alugueres.add(aluguer);  
}
```

Figura xvii: Exemplo intuitivo à explicação.

- **Exceção:** Caso o automóvel já esteja alugado no período solicitado, é lançada uma exceção. Isso evita que múltiplos alugueis sejam feitos para o mesmo carro no mesmo período, mantendo a integridade dos dados.

Método `listarAutomoveisDisponiveis(List<Automovel> todosAutomoveis)`

```
public List<Automovel> listarAutomoveisDisponiveis(List<Automovel> todosAutomoveis) {  
    List<String> alugados = alugueres.stream()  
        .map(aluguer -> aluguer.getAutomovel().getMatricula())  
        .collect(Collectors.toList());  
  
    return todosAutomoveis.stream()  
        .filter(automovel -> !alugados.contains(automovel.getMatricula()))  
        .collect(Collectors.toList());  
}
```

Figura xviii: Método `listarAutomoveisDisponiveis(List<Automovel> todosAutomoveis)`.

Explicação:

- **alugueres.stream().map(...):** Primeiro, criamos uma lista com as matrículas dos carros alugados, usando o método `map()` para extrair a matrícula de cada automóvel associado a um aluguel.
- **filter(...):** Depois, filtramos os automóveis disponíveis, comparando a matrícula de cada carro com as matrículas da lista de carros alugados. Os carros que não estiverem na lista de alugados são considerados disponíveis.
- **Eficiência:** Essa solução é simples e direta, mas pode ser otimizada. A operação de filtragem pode ser mais eficiente se usarmos uma `Set` para armazenar as matrículas alugadas, já que a verificação de presença (`contains()`) num `Set` é geralmente mais rápida do que em uma lista (`List`).

AUTOMOVELSERVICE.JAVA

O `AutomovelService` gerencia o cadastro e manipulação dos automóveis. Vamos detalhar os métodos e discutir como melhorar o código.

Método adicionarAutomovel(Automovel automovel)

```
public void adicionarAutomovel(Automovel automovel) {  
    Optional<Automovel> existente = automoveis.stream()  
        .filter(a -> a.getMatricula().equals(automovel.getMatricula()))  
        .findFirst();  
    if (existente.isPresent()) {  
        throw new IllegalArgumentException("Já existe um automóvel com a matrícula " + automovel.getMatricula());  
    }  
    automoveis.add(automovel);  
}
```

Figura xix: Método `adicionarAutomovel(Automovel automovel)`.

Explicação:

- **Optional<Automovel>:** O método `findFirst()` retorna um `Optional`, que pode conter ou não um valor. O `Optional` ajuda a evitar exceções `NullPointerException` e torna o código mais robusto.
- **if (existente.isPresent()):** Se já houver um automóvel com a mesma matrícula no sistema, uma exceção é lançada, evitando duplicações.
- **Melhoria:** O uso de `Optional` é uma boa prática, mas o método `findFirst()` pode ser substituído por `anyMatch()`, o que pode tornar o código mais semântico:

```
boolean existe = automoveis.stream().anyMatch(a -> a.getMatricula().equals(automovel.getMatricula()));  
if (existe) {  
    throw new IllegalArgumentException("Já existe um automóvel com a matrícula " + automovel.getMatricula());  
}  
automoveis.add(automovel);
```

Figura xx: Exemplo intuitivo da explicação acima.

Método removerAutomovel(String matricula)

```
public void removerAutomovel(String matricula) {  
    automoveis.removeIf(automovel -> automovel.getMatricula().equals(matricula));  
}
```

Figura xxi: Método removerAutomovel(String matricula).

Explicação:

- **removeIf(...):** O método removeIf() é muito eficiente para remover um item da lista com base em uma condição. Nesse caso, remove o automóvel com a matrícula correspondente.
- **Sugestão:** O código está ótimo, mas é importante garantir que a lista de automóveis não contenha carros que estejam associados a alugueres ativos. Caso contrário, seria necessário implementar alguma lógica de verificação antes da remoção.

CLIENTESERVICE.JAVA

O ClienteService gerencia a manipulação dos dados dos clientes.

Método adicionarCliente(Cliente cliente)

```
public void adicionarCliente(Cliente cliente) {  
    Optional<Cliente> existente = clientes.stream()  
        .filter(c -> c.getCc().equals(cliente.getCc()))  
        .findFirst();  
    if (existente.isPresent()) {  
        throw new IllegalArgumentException("Já existe um cliente com o CC " + cliente.getCc());  
    }  
    clientes.add(cliente);  
}
```

Figura xxii: Método adicionarCliente(Cliente cliente).

Explicação:

- **findFirst():** O método `findFirst()` encontra o primeiro cliente com o mesmo CC, garantindo que cada cliente tenha um CC único no sistema. O uso de `Optional` facilita o manejo de casos onde o cliente pode não ser encontrado.

Método `removeCliente(String cc)`

```
public void removeCliente(String cc) {  
    clientes.removeIf(cliente -> cliente.getCc().equals(cc));  
}
```

Figura xxiii: Método `removeCliente(String cc)`.

Explicação:

- **removeIf(...):** Semelhante ao método de remoção de automóveis, este método usa `removeIf` para remover um cliente pela correspondência do número de CC.
- **Possível melhoria:** Antes de remover um cliente, seria interessante verificar se esse cliente tem alugueis pendentes. Caso contrário, a remoção do cliente seria facilitada. Em um sistema real, não seria recomendado remover um cliente com alugueis em aberto, a menos que o sistema tenha um mecanismo de reclassificação ou cancelamento de alugueis.

Conclusão e Melhoria Geral

- **Padrões de Design**
 - **Singleton:** Poderíamos aplicar o padrão Singleton nos serviços se quisermos garantir que exista apenas uma instância de cada serviço (por exemplo, `AluguerService`), controlando melhor o estado da aplicação.
 - **Factory Method:** Poderíamos usar o padrão de projeto Factory para criar instâncias de automóveis, alugueres e clientes, de forma a centralizar a criação desses objetos em um único lugar.

- **Persistência**

Atualmente, os dados estão armazenados apenas em memória. Em um cenário de produção, seria necessário persistir esses dados em uma base de dados (SQL ou NoSQL), o que garantiria que as informações não seriam perdidas ao reiniciar o sistema.

- **Validação e Erros**

Considerar o uso de classes de erro ou mensagens de erro padronizadas pode melhorar a comunicação com os utilizadores. Além disso, o uso de validações mais completas para os dados de entrada pode evitar falhas do sistema.

PASTA UTILS

A pasta utils geralmente contém classes que implementam funcionalidades auxiliares ou genéricas, utilizadas em diferentes partes do sistema. Essas classes são projetadas para melhorar a reutilização e evitar a repetição de código.

No caso de InputUtils, ela abstrai a interação com o utilizador por meio do console, garantindo validações e mensagens de erro claras.

Classe InputUtils

A classe oferece métodos utilitários para ler diferentes tipos de dados do utilizador, além de fornecer feedback imediato em caso de entrada inválida.

Atributos

```
private static final Scanner scanner = new Scanner(System.in);  
private static final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
```

Figura xxiv. Atributos da Classe InputUtils.

scanner:

- É um objeto estático da classe Scanner, usado para capturar a entrada do console.
- A declaração como static garante que apenas uma instância será usada em toda a aplicação.

dateFormatter:

- Formato padrão para interpretar datas no formato dd/MM/yyyy.
- Facilita o uso consistente de datas no sistema.

Métodos: lerString(String mensagem)

- Solicita ao utilizador uma entrada de texto.

Implementação:

```
public static String lerString(String mensagem) {  
    System.out.print(mensagem);  
    return scanner.nextLine();  
}
```

Figura xxv: Implementação dos métodos referidos acima.

Funcionamento:

- Exibe a mensagem recebida como parâmetro.
- Lê e retorna a linha de texto inserida pelo utilizador.
- Exemplo de uso:

```
String nome = InputUtils.lerString("Digite seu nome: ");
```

Figura xxvi: Exemplo do uso do lerString.

lerInt(String mensagem)

- Lê um número inteiro, com validação para evitar erros de entrada.

Implementação:

```
public static int lerInt(String mensagem) {  
    while (true) {  
        try {  
            System.out.print(mensagem);  
            return Integer.parseInt(scanner.nextLine());  
        } catch (NumberFormatException e) {  
            System.out.println("Valor inválido. Tente novamente.");  
        }  
    }  
}
```

Figura xxvii: Implementação de lerInt.

Funcionamento:

- Exibe a mensagem recebida.
- Tenta converter a entrada para int. Caso falhe, exibe uma mensagem de erro e solicita uma nova entrada.

Exemplo de uso:

```
int idade = InputUtils.lerInt("Digite sua idade: ");
```

Figura xxviii: Exemplo do uso de lerInt

lerDouble(String mensagem)

- Similar ao método lerInt, mas captura números decimais.

Implementação:

```
public static double lerDouble(String mensagem) {  
    while (true) {  
        try {  
            System.out.print(mensagem);  
            return Double.parseDouble(scanner.nextLine());  
        } catch (NumberFormatException e) {  
            System.out.println("Valor inválido. Tente novamente.");  
        }  
    }  
}
```

Figura xxix: Implementação do lerDouble.

- **Exemplo de uso:**

```
double salario = InputUtils.lerDouble("Digite seu salário: ");
```

Figura xxx: Exemplo de uso do lerDouble.

lerData(String mensagem)

- Lê uma data no formato dd/MM/yyyy, validando a entrada do utilizador.

Implementação:

```
public static LocalDate lerData(String mensagem) {  
    while (true) {  
        try {  
            System.out.print(mensagem);  
            return LocalDate.parse(scanner.nextLine(), dateFormatter);  
        } catch (DateTimeParseException e) {  
            System.out.println("Formato de data inválido. Utilize o formato dd/MM/yyyy.");  
        }  
    }  
}
```

Figura xxxi: Implementação do lerData.

Funcionamento:

- Exibe a mensagem e tenta converter a entrada para um objeto LocalDate usando o dateFormatter.

- Caso o formato seja inválido, solicita uma nova entrada.

- **Exemplo de uso:**

```
LocalDate dataNascimento = InputUtils.lerData("Digite sua data de nascimento (dd/MM/yyyy): ");
```

Figura xxxii: Funcionamento do lerData.

fecharScanner()

- Fecha o objeto scanner para liberar os recursos.

Implementação:

```
public static void fecharScanner() {  
    scanner.close();  
}
```

Figura xxxiii: Implementação do fecharScanner().

Quando usar:

- Deve ser chamado ao final da execução do programa para evitar vazamento de recursos.

- Exemplo de uso:

```
InputUtils.fecharScanner();
```

Figura xxxiv: Exemplo de uso de fecharScanner().

Análise e Melhorias**Consistência nas mensagens:**

- É possível padronizar as mensagens de erro para facilitar a leitura do utilizador, como adicionar um prefixo comum.

Flexibilidade no formato de data:

- Permitir que o formato de data seja configurável:

```
public static void configurarDateFormatter(String padrao) {  
    dateFormatter = DateTimeFormatter.ofPattern(padrao);  
}
```

Figura xxxv: Flexibilidade no formato da data.

Adaptação para outras fontes de entrada:

- Atualmente, a classe funciona apenas com o console. Para flexibilizar, poderia aceitar um InputStream como parâmetro.

Exceções personalizadas:

- Criar exceções específicas para capturar erros de entrada.

Estabilidade:

- Incluir métodos para simular entradas em testes unitários, como:

```
public static void simularEntrada(String entrada) {  
    System.setIn(new ByteArrayInputStream(entrada.getBytes()));  
}
```

Figura xxxvi: Incluir métodos para simular entradas em testes unitários.

Impacto no Projeto

- **Centralização:** Com InputUtils, todos os métodos de leitura seguem um padrão consistente.
- **Manutenção:** Qualquer alteração na lógica de entrada ou mensagens de erro precisa ser feita apenas nesta classe.
- **Segurança:** A validação reduz a probabilidade de erros ou falhas inesperadas durante a execução.

O arquivo Main.java é o ponto de entrada do sistema. Ele implementa o loop principal e oferece uma interface interativa para o utilizador, organizando as operações relacionadas a automóveis, clientes e alugueres em menus intuitivos.

ANÁLISE DO ARQUIVO MAIN.JAVA

O código é bem estruturado, modularizando cada funcionalidade em métodos separados (gerirAutomoveis, gerirClientes, gerirAlugueres, e mostrarListagens). Vamos explorar as principais secções.

Importações

```
import models.Aluguer;  
import models.Automovel;  
import models.Cliente;  
import services.AluguerService;  
import services.AutomovelService;  
import services.ClienteService;  
import utils.InputUtils;
```

Figura xxxvii: Importações dos modelos.

- Importa as classes dos pacotes models, services, e utils.
- Garante que o Main.java tem acesso a todas as funcionalidades implementadas.

Estrutura Principal

```
public static void main(String[] args) {  
    AutomovelService automovelService = new AutomovelService();  
    ClienteService clienteService = new ClienteService();  
    AluguerService aluguerService = new AluguerService();  
}
```

Figura xxxviii:

- Instancia os serviços necessários para manipular dados de **automóveis**, **clientes** e **alugueres**.
- Esses objetos são passados como parâmetros para os métodos auxiliares, seguindo o princípio de **injeção de dependência**.

Menu Principal

```
while (true) {  
    System.out.println("\n=== Rent Car Lda ===");  
    System.out.println("1. Gerir Automóveis");  
    System.out.println("2. Gerir Clientes");  
    System.out.println("3. Gerir Alugueres");  
    System.out.println("4. Listagens");  
    System.out.println("5. Sair");  
  
    int opcao = InputUtils.lerInt("Escolha uma opção: ");
```

Figura xxxix: Menu Principal do output.

- Exibe o menu principal e captura a escolha do utilizador.
- **Pontos positivos:**
- Clareza na exibição das opções.
- Validação automática com o InputUtils.lerInt.

```
switch (opcao) {  
    case 1 -> gerirAutomoveis(automovelService);  
    case 2 -> gerirClientes(clienteService);  
    case 3 -> gerirAlugueres(automovelService, clienteService, aluguerService);  
    case 4 -> mostrarListagens(automovelService, aluguerService);  
    case 5 -> {  
        System.out.println("Encerrando o sistema...");  
        InputUtils.fecharScanner();  
        return;  
    }  
    default -> System.out.println("Opção inválida. Tente novamente.");  
}
```

Figura xl: Opções do Menu.

- O uso do switch com case simplifica a execução das operações principais.
- Garante a continuidade do loop, exceto no caso de saída (case 5).

Métodos Auxiliares: `gerirAutomoveis(AutomovelService automovelService)`

- Realiza as operações básicas sobre automóveis:
- Adicionar
- Atualizar
- Remover
- Listar
- Exemplo de adição de automóvel:

```
String marca = InputUtils.lerString("Marca: ");
String modelo = InputUtils.lerString("Modelo: ");
String cor = InputUtils.lerString("Cor: ");
int cilindrada = InputUtils.lerInt("Cilindrada: ");
int ano = InputUtils.lerInt("Ano de aquisição: ");
String matricula = InputUtils.lerString("Matrícula: ");
double valorDia = InputUtils.lerDouble("Valor por dia: ");
Automovel automovel = new Automovel(marca, modelo, cor, cilindrada, ano, matricula, valorDia);
automovelService.adicionarAutomovel(automovel);
System.out.println("Automóvel adicionado com sucesso!");
```

Figura xli: Métodos Auxiliares: `gerirAutomoveis(AutomovelService automovelService)`.**Validação explícita:**

- Usa métodos do `InputUtils` para garantir que entradas incorretas são tratadas.
- Para ações como *atualizar* ou *remover*, verifica a existência do automóvel antes de continuar.

`gerirClientes(ClienteService clienteService)`

- Segue a mesma lógica do método anterior, mas aplicada a clientes.
- Exemplo de atualização de cliente:

```
String cc = InputUtils.lerString("CC do cliente a atualizar: ");
clienteService.buscarPorCC(cc).ifPresentOrElse(cliente -> {
    String novoNome = InputUtils.lerString("Novo Nome: ");
    String novaMorada = InputUtils.lerString("Nova Morada: ");
    String novaCartaConducao = InputUtils.lerString("Nova Carta de Condução: ");

    cliente.setNome(novoNome);
    cliente.setMorada(novaMorada);
    cliente.setCartaConducao(novaCartaConducao);

    System.out.println("Cliente atualizado com sucesso!");
}, () -> System.out.println("Cliente não encontrado."));
```

Figura xlii: `gerirClientes(ClienteService clienteService)`.

gerirAlugueres(AutomovelService automovelService, ClienteService clienteService, AluguerService aluguerService)

- Concentra operações específicas sobre alugueres:
- Adicionar um novo aluguer:
- Valida a existência do automóvel e do cliente.
- Trata exceções no construtor de Aluguer.
- Remover aluguer:
- Lista alugueres existentes para o utilizador seleccionar.
- Listar alugueres:
- Exibe todos os alugueres cadastrados.

mostrarListagens(AutomovelService automovelService, AluguerService aluguerService)

- Apresenta relatórios de interesse, como:
- Automóveis alugados:

```
List<Aluguer> alugados = aluguerService.listarAlugueres();
alugados.forEach(aluguer -> System.out.println(aluguer.getAutomovel()));
```

Figura xliii: Automóveis alugados.

- Automóveis disponíveis:

```
List<Automovel> disponiveis = aluguerService.listarAutomoveisDisponiveis(todosAutomoveis);
disponiveis.forEach(System.out::println);
```

Figura xliv: Automóveis disponíveis.

- Alugueres ordenados por data de entrega:

```
List<Aluguer> ordenados = aluguerService.listarAutomoveisAlugadosOrdenadoPorEntrega();
ordenados.forEach(System.out::println);
```

Figura xlv: Alugueres ordenados por data de entrega.

Análise Crítica

1. Clareza e Modularidade:

- Cada funcionalidade é bem separada em métodos específicos.
- O uso de serviços (AutomovelService, etc.) encapsula a lógica de negócios.

2. Validação e Feedback:

- O sistema lida com erros de entrada de maneira eficiente.
- Mensagens claras orientam o utilizador em caso de erros ou opções inválidas.

3. Manutenibilidade:

- Alterações em funcionalidades específicas não afetam o restante do código, graças à modularização.

Sugestões de Melhorias

1. Encapsulamento do Menu Principal:

- Mover o menu para um método dedicado:

```
private static int exibirMenuPrincipal() {  
    System.out.println("\n=== Rent Car Lda ===");  
    System.out.println("1. Gerir Automóveis");  
    System.out.println("2. Gerir Clientes");  
    System.out.println("3. Gerir Alugueres");  
    System.out.println("4. Listagens");  
    System.out.println("5. Sair");  
    return InputUtils.lerInt("Escolha uma opção: ");  
}
```

Figure 44 : Melhorias para o menu.

2. Relatórios Adicionais:

- Estatísticas como número de alugueres por cliente ou receita total.

3. Tratamento de Exceções:

- Capturar exceções inesperadas no método main:

```
try {  
    // loop principal  
} catch (Exception e) {  
    System.out.println("Erro inesperado: " + e.getMessage());  
}
```

Figure 45: Capturar exceções inesperadas no método main.

4. Logs:

- Adicionar logs para acompanhar ações importantes, útil para auditoria e depuração.

5. Testes Unitários:

- Criar métodos simulados para entradas do usuário, permitindo testes automáticos dos menus.

CONCLUSÃO

Este projeto, desenvolvido no contexto da unidade curricular de Linguagens e Teoria da Computação, representa um esforço consolidado para aplicar conceitos teóricos e práticos na construção de um sistema robusto e funcional para a gestão de aluguer de automóveis. Mediante uma abordagem estruturada e modular, foi possível implementar um software que atende aos requisitos de gestão de automóveis, clientes e alugueres, com um foco particular em boas práticas de programação e eficiência na manipulação de dados.

A organização modular do sistema, com a separação clara entre os pacotes *models*, *services* e *utils*, garantiu um design coeso e alinhado ao padrão de arquitetura MVC. Essa estrutura facilitou tanto a extensibilidade quanto a manutenção do código, assegurando que futuras funcionalidades possam ser integradas sem comprometer a estabilidade do sistema. Entre as principais funcionalidades implementadas estão a gestão completa de automóveis, incluindo adição, atualização, remoção e listagem; a gestão de clientes, com processos análogos aos dos automóveis; e a gestão de alugueres, que permite a atribuição de automóveis a clientes com validação de disponibilidade e cálculo automático das condições de aluguer. Além disso, foram desenvolvidas listagens específicas, como relatórios de automóveis alugados e disponíveis, proporcionando ao utilizador informações detalhadas para a tomada de decisões.

A interface interativa foi projetada para ser intuitiva, com um menu simples suportado por métodos de entrada validados do *InputUtils*. Esse componente assegurou uma experiência fluida ao utilizador, prevenindo falhas inesperadas por meio de um tratamento robusto de erros na entrada de dados. No núcleo do desenvolvimento, a orientação a objetos foi utilizada de forma estratégica, com cada classe do pacote *models* projetada para garantir encapsulamento, coerência de atributos e validação interna, o que reduziu erros e promoveu a reutilização do código. O uso de boas práticas de programação foi evidente, desde o emprego de padrões de design como o *Singleton* para gerir instâncias de serviços até a separação clara de responsabilidades, assegurando a qualidade técnica do projeto.

Apesar dos avanços, o desenvolvimento apresentou desafios, como a validação de regras de negócio para garantir que as operações respeitassem as restrições de disponibilidade de automóveis e a validade de datas. A integração entre os componentes

exigiu um cuidado especial para gerir dependências de forma eficiente. Além disso, o design do fluxo de utilizador, especialmente na estruturação do menu principal e os seus submenus, foi constantemente refinado para oferecer uma navegação acessível.

Ao longo do projeto, várias lições foram aprendidas. A importância de um planeamento estrutural ficou clara, especialmente na divisão inicial dos pacotes que simplificou a implementação e reduziu a complexidade. A abstração e reutilização de código, como observado nos métodos genéricos do *InputUtils*, economizaram tempo e evitaram redundância. A implementação de testes e validações robustas foi fundamental para garantir a integridade do sistema, especialmente em entradas de dados e regras de negócios. A colaboração sob a orientação do professor José Arnaud foi essencial para alinhar o projeto às normas exigidas e oferecer insights valiosos sobre a aplicação de conceitos teóricos, na prática.

Embora o sistema já apresente funcionalidades sólidas, há diversas perspetivas de melhorias e extensões. Entre elas, está a integração com um banco de dados relacional para persistência de informações, a evolução para uma interface gráfica utilizando bibliotecas como *JavaFX* ou *Swing*, a inclusão de relatórios avançados que analisem dados financeiros e o comportamento de clientes, e até mesmo a integração com APIs externas para funcionalidades como o cálculo de distâncias percorridas ou estimativas de custos de viagem.

O desenvolvimento deste sistema foi uma excelente oportunidade para aplicar conceitos fundamentais da Engenharia Informática, como programação orientada a objetos, design de software e validação de entradas. O projeto reflete não apenas a capacidade técnica adquirida ao longo do curso, mas também um entendimento profundo das necessidades e desafios de um sistema real de gestão. O resultado final é um produto funcional, bem documentado e alinhado às boas práticas de engenharia de software. Ele representa um marco importante na trajetória académica e profissional, evidenciando o compromisso com a qualidade e a inovação, e servindo como uma base sólida para evoluções futuras, seja em contextos académicos ou empresariais.

BIBLIOGRAFIA

Arraylist Java: Explorando a classe ArrayList no Java. (2012, April 30). DevMedia. <https://www.devmedia.com.br/explorando-a-classe-arraylist-no-java/24298>

Arraylist Java: Usando listas Em Java! (2019, July 20). Insights para te ajudar na carreira em tecnologia | Blog da Trybe. <https://blog.betrybe.com/java/arraylist-java/>

Cache://rumos.pt/formacao/explorador-de-cursos/academia-programador-java?gad_source=1 - Google search. (2017, May 23). Rumos, Formação e Consultoria em Tecnologias da Informação. https://rumos.pt/formacao/explorador-de-cursos/academia-programador-java?gad_source=1&gbraid=0AAAAADKSloqSXi83UUf9tS0Qt2c_UoIqG&gclid=CjwKCAiAmfq6BhAsEiwAX1jsZ1ewJbKiWxrIUPySOUYD7aHRvKmC7Add592revulLvI7SGIdWWaSwRoCkWMQAvD_BwE

Classes and objects in Java. (2023, September 22). GeeksforGeeks. <https://www.geeksforgeeks.org/classes-objects-java/>

Declaração E utilização de classes no Java. (2017, May 31). DevMedia. <https://www.devmedia.com.br/java-declaracao-e-utilizacao-de-classes/38374>

Java class and objects (With example). (2011, April 10). Programiz: Learn to Code for Free. <https://www.programiz.com/java-programming/class-objects>

Java documentation. (2018, July 30). Oracle Help Center. <https://docs.oracle.com/en/java/>

Java: Trabalhando com listas E coleções de dados. (2024, November 22). Alura. https://www.alura.com.br/conteudo/java-listas-colecoes-dados?srsId=AfmBOor_XlvTE1r-pXesg3dIJ4sGMBgCVccgei_nGCFAKDp1Isu7NnxK

Lista Java. (2012, November 6). CodeGym. <https://codegym.cc/pt/groups/posts/pt.813.lista-java>

Listas Em Java. (2023, August 19). Caffeine Algorithm. <https://caffeinealgorithm.com/blog/listas-em-java>

Listas Em Java: Uma Introdução simples E Prática. (2023, September 18). DEV Community. <https://dev.to/antoniorws/listas-em-java-uma-introducao-simples-e-pratica-2e79>

Lobo, B. (2024, July 27). *Arrays E Listas Em Java - Como finalmente aprender a USA-las!* DIO. <https://www.dio.me/articles/arrays-e-listas-em-java-como-finalmente-aprender-a-usa-las>

Silva, T. (2023, August 29). *Introdução as Listas Em Java E Suas Implementações.* DIO. <https://www.dio.me/articles/introducao-as-listas-em-java-e-suas-implementacoes>

ANEXOS

- Código-Fonte Completo:
 - Localização: Disponível na pasta Código/GestaoAluguerFrota;
 - Descrição: Inclui todas as classes, serviços e utilitários utilizados no desenvolvimento da aplicação.